Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity. (GUI)

```
import tkinter as tk
from tkinter import messagebox
import hmac
import hashlib

def generate_mac(message, key):
    return hmac.new(key.encode(), message.encode(),
hashlib.sha256).hexdigest()

def verify_mac(message, key, mac):
    expected = generate_mac(message, key)
    return hmac.compare_digest(expected, mac)

def do_generate():
    if not msg_var.get() or not key_var.get():
        messagebox.showerror("Error", "Message and key required")
        return
    out.set(generate_mac(msg_var.get(), key_var.get()))

def do_verify():
    if not msg_var.get() or not key_var.get() or not mac_var.get():
        messagebox.showerror("Error", "Message, key and MAC required")
        return
    result = verify_mac(msg_var.get(), key_var.get(), mac_var.get())
    messagebox.showinfo("Result", "Valid MAC" if result else "Invalid
MAC")

root = tk.Tk()
root.title("MAC Generator/Verifier - GUI")

msg_var = tk.StringVar()
key_var = tk.StringVar()
mac_var = tk.StringVar()
out = tk.StringVar()

tk.Label(root, text="Message").grid(row=0, column=0, padx=5, pady=5)
```

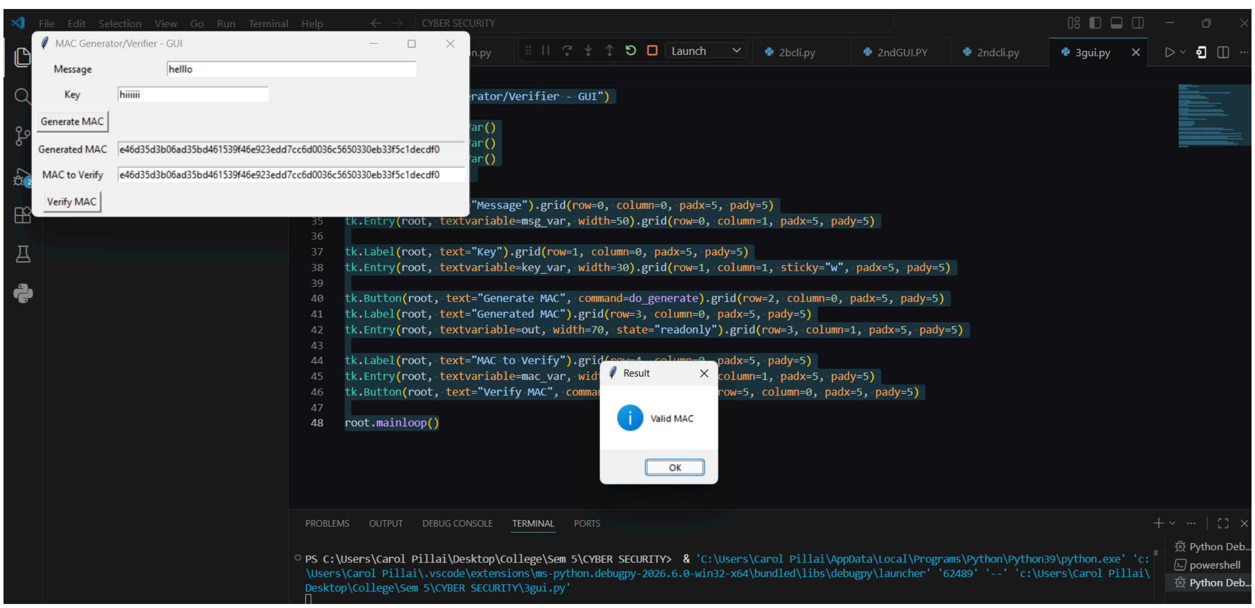
```
tk.Entry(root, textvariable=msg_var, width=50).grid(row=0, column=1,
padx=5, pady=5)

tk.Label(root, text="Key").grid(row=1, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=key_var, width=30).grid(row=1, column=1,
sticky="w", padx=5, pady=5)

tk.Button(root, text="Generate MAC", command=do_generate).grid(row=2,
column=0, padx=5, pady=5)
tk.Label(root, text="Generated MAC").grid(row=3, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=out, width=70, state="readonly").grid(row=3,
column=1, padx=5, pady=5)

tk.Label(root, text="MAC to Verify").grid(row=4, column=0, padx=5, pady=5)
tk.Entry(root, textvariable=mac_var, width=70).grid(row=4, column=1,
padx=5, pady=5)
tk.Button(root, text="Verify MAC", command=do_verify).grid(row=5,
column=0, padx=5, pady=5)

root.mainloop()
```



Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity. (CLI)

```python
import sys
import hmac
import hashlib

def generate_mac(message, key):
    return hmac.new(key.encode(), message.encode(),
hashlib.sha256).hexdigest()

def verify_mac(message, key, mac):
    expected = generate_mac(message, key)
    return hmac.compare_digest(expected, mac)

if __name__ == "__main__":
    if len(sys.argv) < 4:
        print("Usage: python 2B_mac_cli.py generate <message> <key>")
        print("       python 2B_mac_cli.py verify <message> <key> <mac>")
    else:
        mode = sys.argv[1]
        if mode == "generate":
            message, key = sys.argv[2], sys.argv[3]
            print(generate_mac(message, key))
        elif mode == "verify":
            message, key, mac = sys.argv[2], sys.argv[3], sys.argv[4]
            print("Valid MAC" if verify_mac(message, key, mac) else
"Invalid MAC")
        else:
            print("Invalid mode")
```

```
       python 2B_mac_cli.py verify <message> <key> <mac>
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python 2bcli.py generate "helllo" hiiiiiii
Invalid mode
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python 3cli.py generate "helllo" hiiiiiii
09ba42edd8b74a0de4e8f752e1047f3d85dcc4679c525d86aa20ebf83b7b5707
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python 2bcli.py verify "helllo" hiiiiiii 09ba42edd8b74a0de4e8f752e1047f3d85dcc46
79c525d86aa20ebf83b7b5707
Usage: python 1B_transposition_cli.py [encrypt|decrypt] <text> <key>
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY>
Ln 25, Col 34 (894 selected)   Spaces: 4   UTF-8   CRLF   { } Python   Python 3
```